

CSCE 641 COMPUTER GRAPHICS · MODULE 08 · LECTURE NOTES

Texture Mapping, Filtering, and Mipmaps

Putting detail on surfaces without putting geometry there.

MODULE OUTCOMES

- Explain UV parameterisation and the artifacts of a bad one.
- Choose correctly between magnification and minification filters.
- Explain why mipmaps exist in sampling terms, and what trilinear and anisotropic filtering each fix.
- Implement tangent-space normal mapping correctly.
- Name the common texture types and say which are colour and which are data.

1 Parameterisation

A texture is an image; a surface is a 2D manifold embedded in 3D. Texture mapping needs a function from the surface to the image plane, and supplying it per vertex as a (u, v) coordinate, then interpolating, is the standard approach. Module 04 already established that the interpolation must be perspective-correct.

Constructing a good UV map is the problem of flattening a curved surface onto a plane, and it is exactly the problem cartographers have. Only *developable* surfaces — cylinders, cones — flatten without distortion. A sphere does not, which is why every world map distorts something. Three costs trade against each other:

- **Stretch.** Texels cover unequal areas of the surface, so detail density varies visibly.
- **Seams.** Cuts in the UV layout. Across a seam, filtering and mipmapping break down and a visible line appears.
- **Wasted space.** Gaps between UV islands are texture memory that holds nothing.

You cannot eliminate all three. Module 12 returns to this with the geometry-processing tools to reason about it properly.

1.1 Wrap modes

Mode	Outside [0,1]	Typical use
REPEAT	Fractional part; tiles infinitely	Floors, walls, fabric — any tiling detail.
MIRRORED_REPEAT	Tiles with alternate flipping	Hides the seam of a texture that does not tile perfectly.

Mode	Outside [0,1]	Typical use
CLAMP_TO_EDGE	Holds the edge texel	Decals, UI, skyboxes. Prevents bleeding from the opposite edge.
CLAMP_TO_BORDER	Returns a specified constant	Shadow maps (Module 10): a border of 'fully lit' means geometry outside the light's frustum is not spuriously shadowed.

2 Filtering

A texture is a grid of samples. The screen is a different grid of samples. They almost never align, and the two ways they fail to align are qualitatively different problems.

2.1 Magnification

When texels are larger than pixels, several pixels fall inside one texel. Nearest-neighbour sampling produces visible squares; bilinear interpolation between the four nearest texels produces a smooth, slightly soft result. This is a resolution problem: there genuinely is not enough information, and all you can do is interpolate plausibly. It is annoying but not broken, and bilinear is almost always the right answer.

2.2 Minification

When texels are smaller than pixels, many texels fall inside one pixel. The pixel's correct value is the *average* of the texture over its footprint. Point sampling instead picks one texel from that footprint and treats it as representative.

WHY THAT FLICKERS

Move the camera slightly. The footprint shifts, a different texel is selected, and the pixel's colour jumps — with no relation to the actual average, which barely changed.

Across a whole surface this produces shimmering, crawling, and moiré patterns on regular detail like brickwork or a checkerboard. It is at its worst exactly where it is most visible: distant ground planes in motion.

This is undersampling — the same phenomenon as the jagged edges of Module 01 and the vanishing highlights of Module 06. Module 09 develops the theory.

3 Mipmapping

The correct value is an average over the pixel's footprint. Computing that average at render time would require reading every texel in the footprint — potentially thousands. The solution is to precompute the averages.

A **mipmap** is a chain of progressively halved versions of the texture, each level the filtered average of the one above. At render time, select the level at which one texel corresponds to approximately one pixel, and the averaging has already been done offline. The storage cost is the geometric series $1 + 1/4 + 1/16 + \dots = 4/3$: just 33% extra.

3.1 Level selection

$$\rho = \max(|\partial(u,v)/\partial x|, |\partial(u,v)/\partial y|) \quad \text{level} = \log_2 \rho$$

The derivatives are computed from neighbouring threads in the 2×2 quad, which is the mechanism Module 07 described. This is the reason that quad exists, and the reason that sampling a texture inside non-uniform control flow has undefined results: the neighbouring threads may not have executed the sample, so there is nothing to difference against.

Selecting a single integer level produces a visible band where the level changes. **Trilinear** filtering samples the two neighbouring levels bilinearly and interpolates between them, removing the band for eight taps instead of four.

3.2 Anisotropy

Level selection takes the *maximum* of the two derivatives, which is correct only when the pixel's texture-space footprint is roughly square. On a surface seen at a shallow angle it is not: the footprint is a long thin quadrilateral, perhaps one texel across and sixteen long.

Taking the maximum selects a level coarse enough for the long axis, and the short axis is then blurred sixteen times more than necessary. The surface looks correct in the direction it is stretched and smeared across it. This is why untreated floors in older games go mushy in the distance.

Anisotropic filtering instead takes several samples along the long axis of the footprint, at a level appropriate to the *short* axis. Sixteen-tap anisotropic filtering costs four times trilinear and transforms the appearance of ground planes, which is why it is historically the single highest-value graphics setting per unit of performance.

4 Normal mapping

Geometric detail is expensive: more triangles means more vertex work, more memory, and smaller triangles (which Module 07 showed are disproportionately costly because of quad overshading). Normal mapping provides the *appearance* of fine geometric detail without the geometry, by perturbing the normal used for lighting.

Because almost all shading depends on the normal — the cosine law, the specular lobe, the Fresnel term — changing the normal changes the lighting exactly as though the surface had that shape. Bumps, scratches, fabric weave, and skin pores all become texture resolution rather than geometry.

THE ONE THING IT CANNOT DO

The silhouette is unchanged. Viewed edge-on, a normal-mapped brick wall is visibly a flat plane with no bricks protruding.

Parallax occlusion mapping and displacement mapping address this at greater cost. Nothing fixes it for free, and knowing the limit tells you when to spend real geometry.

4.1 Tangent space

A normal map could store normals in object space, but then the map is tied to one specific mesh in one specific pose: it cannot be reused, cannot tile, and is wrong the moment the surface deforms under skinning (Module 13).

Tangent space stores the normal relative to the surface itself: Z along the surface normal, X and Y along the directions in which u and v increase. An unperturbed normal is (0, 0, 1), which encodes to (0.5, 0.5, 1.0) — hence the characteristic lavender blue of normal maps. Such a map is reusable across models, tileable, and correct under arbitrary deformation.

4.2 Building the TBN

```

// Vertex shader
vec3 T = normalize(uNormalMatrix * aTangent.xyz);
vec3 N = normalize(uNormalMatrix * aNormal);

// Gram-Schmidt: interpolation and the normal matrix both cause T to
// drift out of perpendicularity with N. Re-orthogonalise.
T = normalize(T - dot(T, N) * N);

// aTangent.w carries handedness, +1 or -1. Mirrored UV islands have
// the opposite handedness, and without this they light inside out.
vec3 B = cross(N, T) * aTangent.w;

vTBN = mat3(T, B, N);

// Fragment shader
vec3 n = texture(uNormalMap, vUV).rgb * 2.0 - 1.0; // decode [0,1]->[-1,1]
vec3 N = normalize(vTBN * n); // tangent -> world

```

TWO DETAILS THAT ARE FREQUENTLY WRONG

Handedness. Mirroring UV islands is standard practice — a character's two halves share texture space, which halves memory. Mirrored islands have reversed handedness, and omitting the *w* term lights one side of every mirrored object incorrectly. On a face it is subtle and deeply confusing.

Gram-Schmidt. The interpolated tangent is not exactly perpendicular to the interpolated normal, and the normal matrix can shear them further apart. Without re-orthogonalisation the basis is skewed and lighting is subtly wrong across the whole surface.

5 The texture set

A physically based material (Module 11) is typically four to six textures. The critical practical point, carried over from Module 06, is which are colour and which are data.

Map	Contents	Space	Notes
Albedo / base colour	Surface colour, no lighting baked in	sRGB	Must contain no shadows or highlights; those come from lighting.
Normal	Tangent-space normal, encoded	Linear	Tagging this sRGB is the classic subtle bug.
Roughness	Microfacet distribution width	Linear	Often packed into a channel alongside metallic and AO.
Metallic	Conductor vs dielectric	Linear	Physically near-binary; intermediate values usually mean a blend boundary.
Ambient occlusion	Local self-occlusion	Linear	Applies to ambient and indirect light only, never to direct light.
Emissive	Self-emitted light	sRGB	Added after lighting, not multiplied by it.

Map	Contents	Space	Notes
Height	Offset along the normal	Linear	For parallax or tessellation displacement.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
GAMES101 Lectures 09–10 — Texture Mapping, Advanced Texturing	Mipmapping derived as a sampling fix, with excellent diagrams of the anisotropic footprint problem.
LearnOpenGL — Textures, Normal Mapping, Parallax Mapping	Complete working TBN construction including the handedness detail.
Scratchapixel — Texture Mapping	Written treatment with the mip-level derivation in full.
Nathan Reed — 'Understanding BCn Texture Compression Formats'	Why normal maps use BC5 and colour uses BC7, and what each format sacrifices. Short, practical, free.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Add texture sampling to your software rasterizer with nearest and bilinear filtering. Render a textured plane filling the screen with each and capture the difference.
- 2 Build a mipmap chain yourself by repeated box filtering. Implement level selection from UV derivatives and trilinear interpolation between levels.
- 3 Render a large checkerboard floor receding to the horizon with: point sampling, bilinear, trilinear, and $16\times$ anisotropic. Capture all four. Record a short video of each with the camera moving forward — the aliasing is a motion artifact and still images undersell it.
- 4 Implement tangent-space normal mapping on a mesh with mirrored UVs. Render with and without the handedness term and capture the difference on the mirrored half.
- 5 Remove the Gram–Schmidt step and render a mesh with a non-uniform scale. Describe the resulting error.
- 6 Measure the memory cost of your mipmap chain and confirm it is $4/3$ of the base texture.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why can a sphere not be UV-unwrapped without distortion, and what three costs trade against each other in any UV layout?
- 2 Explain the difference between magnification and minification artifacts, and why only one of them is a sampling failure.
- 3 What problem do mipmaps solve, what do they cost in memory, and where does the $4/3$ come from?
- 4 How is the mip level selected, and why does this make texture sampling inside divergent control flow undefined?
- 5 Why does trilinear filtering over-blur a floor seen at a shallow angle, and how does anisotropic filtering address it?
- 6 Give two reasons to store normal maps in tangent space rather than object space, and name the two construction details most often got wrong.